

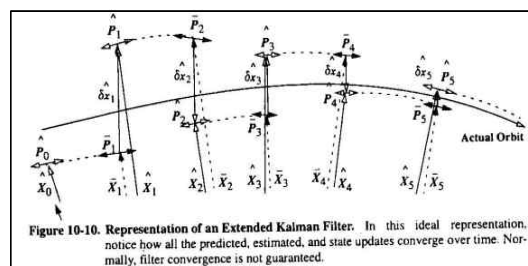
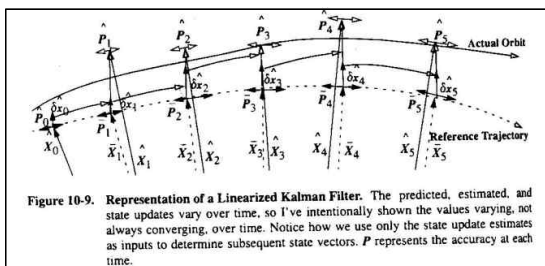
1) Please determine satellite attitude using the EKF method, by following the simulations in the paper below. Include external disturbance torques as much as possible. (Matlab codes will be provided.)

LAB#10은 궤도결정과 필터링 및 위성유도제어 후반부에 다루는 Extended Kalman Filter(EKF)를 이용하여 자세제어를 하는 과제이다. Kalman Filter에 대해 간략하게 설명하기 위해 궤도결정과 필터링 과제에 작성한 내용을 일부 참고하였다.

Kalman Filter는 $\hat{X}_k$ (이전 시각의 state estimate), $\hat{P}_k$ (정확도 나타내는 Covariance Matrix), $z_{k+1}(obs)$ (현재 시각에 관측된 값), Q (Dynamics와의 Error), R_{k+1} (현재 시각 관측값의 error) 등의 조건이 주어지면 그 다음 시각의 $\hat{X}$ 와 $\hat{P}$ 를 알 수 있게해주는 측정 기법이다. Kalman Filter의 장점은 계산이 빠르고 요구되는 메모리 용량이 적으며, 현재 시각에 실시간으로 State와 Covariance Matrix를 추정할 수 있다는 점이 있다. 또한, Batch Filter와 달리 시간이 변화하는 것을 고려하여 실시간으로 계산하므로 매 시간마다 변화하는 변수들을 반영할 수 있다. Kalman Filter의 종류에는 세 가지가 있으며 LKF(Linear Kalman Filter), EKF(Extended Kalman Filter), UKF(Unscented Kalman Filter)이다.

가장 먼저, LKF(Linear Kalman Filter)는 특정 State를 기준으로 하여 새로운 Reference trajectory를 만들어 내며 이를 바탕으로 propagation을 한다. Reference trajectory와 actual orbit 간의 차이를 계산하는 방식으로 위치를 추정하게 되며 첫 state부터 propagation을 이어 나가기 때문에 시간이 지남에 따라 오차가 누적될 수 있다. 따라서 최종 궤도를 계산하였을 때 actual orbit과의 차이가 많이 발생할 수 있다는 치명적인 단점이 있다. 이 단점을 보완하기 위해 사용하는 Filtering 방식이 EKF(Extended Kalman Filter)이다.

EKF에서는 관측된 지점마다 새로운 Trajectory를 생성하는 방식을 이용하여 오차가 누적되는 것을 방지할 수 있다. LKF에서의 $\hat{X}$ 와 $\hat{P}$ 는 $\delta x_{k+1}=0$ 을 기준으로 하지만 EKF에서는 현재 시각에 대해 update하며 매번 새로운 Reference Trajectory를 계산할 수 있다. 일련의 과정에 차이가 있음을 이해하기 위해 두 사진을 첨부하여 비교해보았다. (순서대로 LKF, EKF)



본격적으로 자세제어 simulation을 하기에 앞서, 문제에 제시된대로 External disturbance torque를 최대한 이용하기 위해 첨부된 matlab 파일을 참고해보았다. Torque에 포함되는 Matlab 코드에는 gravity gradient 및 lab#9에서도 다루었던 지구 자기장에 의한 벡터를 나타낸 BDipole.m, 태양의 position 벡터를 나타낸 SunV1.m 파일 등이 있었고 그 외에도 여러 코드들이 폴더에 포함되어 있었다. 그 중 Kalman Filter를 이용한 자세결정에 대한 내용이 담겨있는 main.m 코드를 가장 중점적으로 활용할 수 있었다.

```

5
6 %% Initial setting
7 constant % load constants
8
9 tstep = 10;
10 tf = 6000;
11 % tf = 5781;
12 t = 0 : tstep : tf-1; % Simulation time
13 n = length(t);
14
15 jD = JD2000 + t/86400; % Julian date
16
17 r_true = zeros(3, n);
18 v_true = zeros(3, n);
19 C_true = zeros(3,3,n);
20 X_true = zeros(7,n);
21 Euler_true = zeros(3,n);
22
23 %% Orbit & Attitude Propagation (true value)
24
25 for i = 1 : n
26     if i == 1
27         r_true(:,1) = r0;
28         v_true(:,1) = v0;
29         X_true(:,1) = X_i; % initial position, velocity, and state preallocation
30     else
31         [r_true(:,i),v_true(:,i)] = pkepler(r_true(:,i-1), v_true(:,i-1), t(i)-t(i-1), 0, 0);
32         % propagate orbit using pkepler function, find true position and velocity
33
34         [~, X_temp] = ode45(@(t,x) dynamics(t,x, r_true(:,i)), [t(i-1),t(i)], X_true(:,i-1));
35         % propagate state solving differential equations
36
37         X_true(:,i) = X_temp(end,:);
38         X_true(1:4,i) = X_true(1:4,i) / norm(X_true(1:4,i)); % normalize quaternions
39     end
40     C_true(:,i,i) = quat2rotm(X_true(1:4,i)); % true rotation matrix
41     Euler_true(:,i) = rotm2eul(C_true(:,i,i)); % true euler angles
42 end
43
44 %% Measurement data in ECI frame
45
46 r_hat = r_true + normrnd(0, err_prop, size(r_true));
47 % Measured position with Gaussian noise of propagation
48 v_hat = v_true;
49
50 B_obs = BDipole(r_hat,jD) + normrnd(0, err_IGRF, size(r_hat));
51 % Measured B-field with IGRF noise of 20 nT
52 S_hat_temp = SunV1(jD,r_hat); % Sun vector without any error
53

```

시뮬레이션 및 Julian Date에 대한 시간정보의 초기 설정 값

궤도 및 자세 propagation에 대한 상수 (초기 r,v,X 등)

ECI 좌표계에서의 관측 데이터에 대한 정보

Gaussian noise의 관측값 및 일정 noise에서의 지구 자기장 함수 noise를 제외한 Sunvector의 함수

작성된 내용은 위와 같이 초기 설정 값과 궤도 및 자세 propagation에 대한 함수들이 있었고 특히 ECI 좌표계에서의 관측 데이터가 포함된 함수에는 이전 과제에서도 공부했던 B-field 및 Sun-vector에 의한 error를 다루었다. 지구 자기장에 의한 vector는 20nT의 일정 noise가 포함되었지만 sun-vector에는 아무런 noise가 포함되지 않은 값에서 시작하였다.

```

55
56 SUN_err_temp = rand(3,n); % generate 3 random number for Euler principle axis
57 SUN_err = zeros(3, n);
58 SUN_err_angle = zeros(1,n);
59 SUN_err_q = zeros(4, n);
60 SUN_err_rotmat = zeros(3,3,n);
61 S_obs = zeros(3,n);
62
63 for i = 1 : n
64     SUN_err(:,i) = Unit(SUN_err_temp(:,i)); % normalize Euler principle axis
65     SUN_err_angle(:,i) = normrnd(0, err_Sangle);
66     % generate Euler principle angle error using Sun-Vector measurement noise
67
68     SUN_err_q(:,i) = [cos(SUN_err_angle(:,i)/2) SUN_err(1,i)*sin(SUN_err_angle(:,i))...
69                     SUN_err(2,i)*sin(SUN_err_angle(:,i)) SUN_err(3,i)*sin(SUN_err_angle(:,i))];
70     % Make quaternion reflecting Sun-Vector measurement noise
71     SUN_err_rotmat(:,i) = quat2rotm(SUN_err_q(:,i));
72     S_obs(:,i) = SUN_err_rotmat(:,i) * S_hat_temp(:,i);
73     % New sun-vector including Sun-vector measurement noise
74 end

```

Euler principal axis와 3축에 해당하는 난수를 발생

생성된 principal Euler angle을 이용하여 rotation matrix를 계산

이를 normalize 한다

새롭게 생성된 noise가 포함된 sun-vector

그 이후에 위와 같이 Sun_error를 결정하는데 우선 modeling된 Sun-vector의 noise에서 Euler principal axis의 세 축에 해당하는 난수 3개를 생성하고 이를 normalize 작업을 한다. 그리고 제시된 각도에 해당하는 principal Euler angle을 만들고 이에 대한 rotation matrix를 통해 error 즉, 잡음(noise)이 포함된 Sun-vector를 구할 수 있다.

```

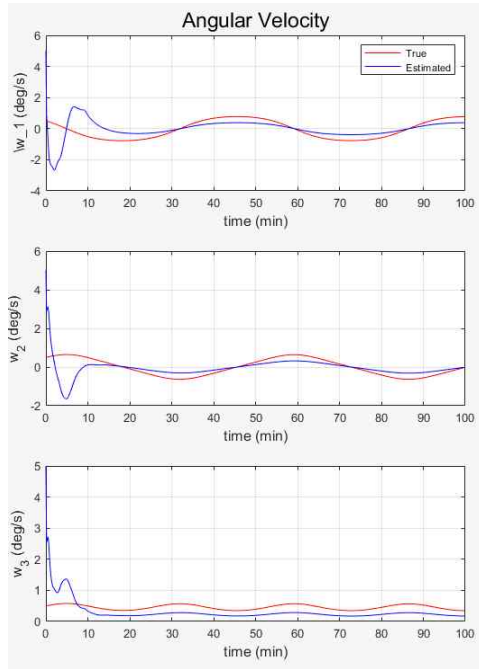
76 %% Filtering
77
78 X_hat = zeros(7,n);
79 X_bar = zeros(7,n);
80 Euler_hat = zeros(3,n);
81 C_hat = zeros(3,3,n);
82 z = zeros(6,n);
83 P_hat = zeros(7,7,n);
84 P_bar = zeros(7,7,n);
85 K = zeros(7,6,n);
86 % preallocation
87
88 X_hat(:,1) = [0 1/sqrt(3) 1/sqrt(3) 1/sqrt(3) 5*rad 5*rad 5*rad]';
89 % Estimated initial state, starts from 180 degree attitude error
90 % 10 times larger angular velocity estimation
91 P_hat(:,1) = (X_hat(:,1)-X_true(:,1))*(X_hat(:,1)-X_true(:,1))';
92 % initial error covariance matrix
93
94 Q = diag([1 1 1 1]*1e-5 [1 1 1]*1e-10);
95 R = 1e-3.*eye(6);
96 % Design parameter gain Q & R. Select each value empirically

```

X,C,P 등, 함수의 size를 표현

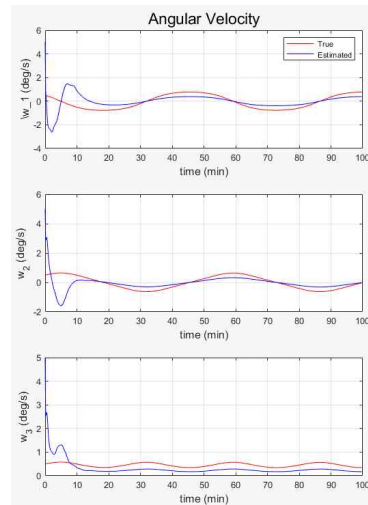
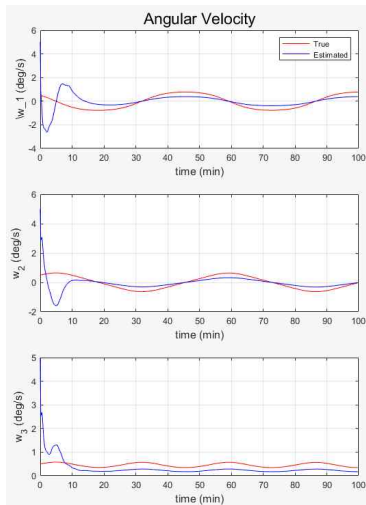
이후 76부터는 본격적으로 Filtering 함수에 대한 설명이 나와 있었는데 180도부터의 초기 estimate 값이 설정되어 있음을 볼 수 있다. 주어진 코드에 따라 각속도는 실제 값의 10배를 사용하여 simulation 한다. 이를 통해 계산된 수렴 정도를 확인 후 초기 값 설정 여부를 판단해볼 수 있다. 마지막으로 EKF의 변수 중 Dynamics와의 Error 정도를 나타내는 $Q(E[\nu\nu^T])$ 와 현재 시각의 관측 값 error를 나타내는 $R(W^{-1})$ 를 계산하면 filtering이 가능하다. 수업시간에 배운 EKF의 과정에 대한 내용을 참고했으며 그 이후 K(Kalman gain) 또한 코드에 적힌 함수관계를 통해 얻을 수 있었다.

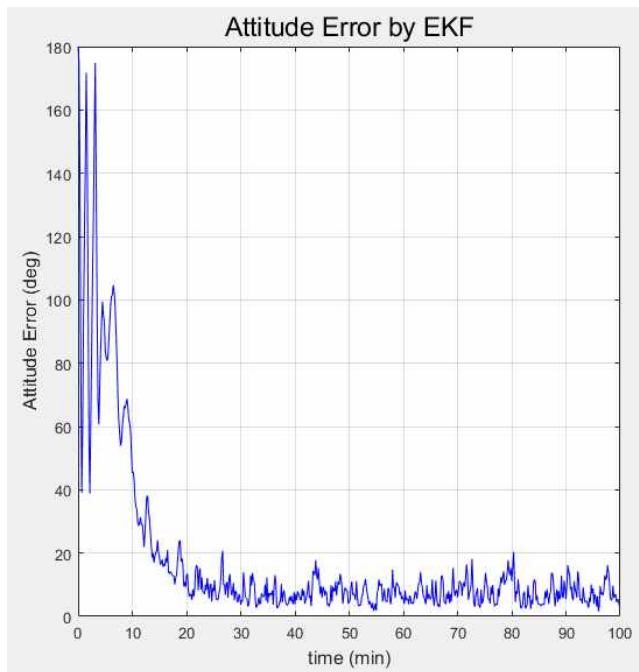
위에서 설명한 main.m 파일을 실행한 결과를 살펴보기로 했다. 이와 연관된 constant.m 파일에는 여러 가지 외부 요인에 의한 상수가 입력되어 있었고 dynamics.m 파일에는 gravity gradient에 의한 torque의 정보가 입력되어 있었다. 최대한 많은 external disturbance torque를 포함시키기 위하여 모든 상수를 default 값으로 두었다. 가장 처음으로 논문에 언급된 초기 자세 error 180°와 solar cell noise의 값 $\sigma_{solar}=15^\circ$ (table 1)를 constant.m 파일에 적용한 후 실행해보았다.



이에 대한 값으로 실행한 결과, angular velocity에서는 실제 값과 추정 값 사이에서 다음과 같은 차이를 보였다. 순서대로 Roll, Pitch, Yaw angle이며 세 방향에는 어느 정도 차이가 있었지만 처음 몇 분 동안은 추정된 값이 실제 값과 큰 차이를 보였으나 일정 시간이 지난 후 (Roll : 30min, Pitch : 20min, Yaw : 8~9min) 비슷한 경향을 보이게 된다.

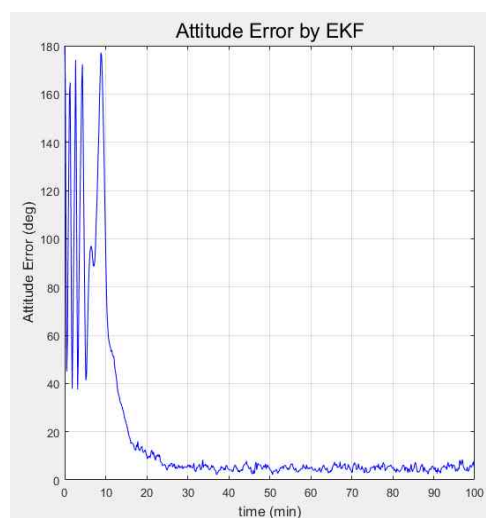
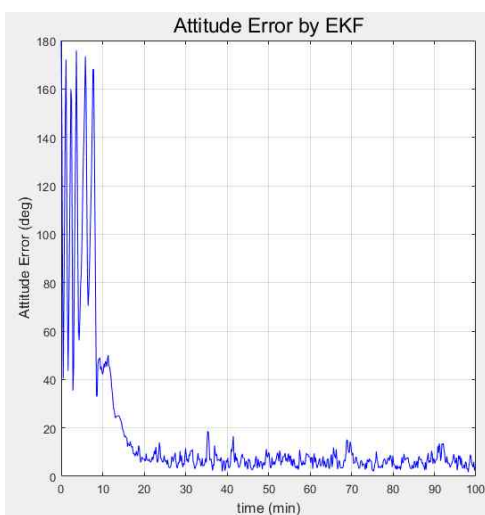
이러한 오차발생을 줄이기 위해 논문에서와 마찬가지로 σ_{solar} 값을 조정하면서 실행을 해보았다. $\sigma_{solar}=10^\circ$ 일 때와 $\sigma_{solar}=5^\circ$ 일 때의 결과를 살펴보니 다음과 같이 위의 결과와 별다른 차이를 보이지 않았고 따라서 solar cell noise의 변화만으로는 angular velocity에 크게 영향을 주지 않는다는 것으로 판단해보았다. (왼쪽부터 순서대로 $\sigma_{solar}=10^\circ$, $\sigma_{solar}=5^\circ$)

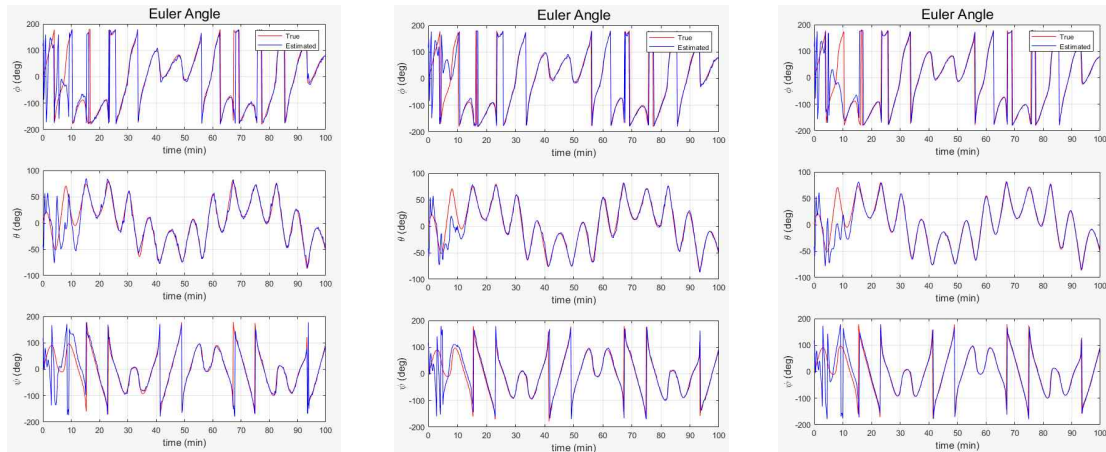




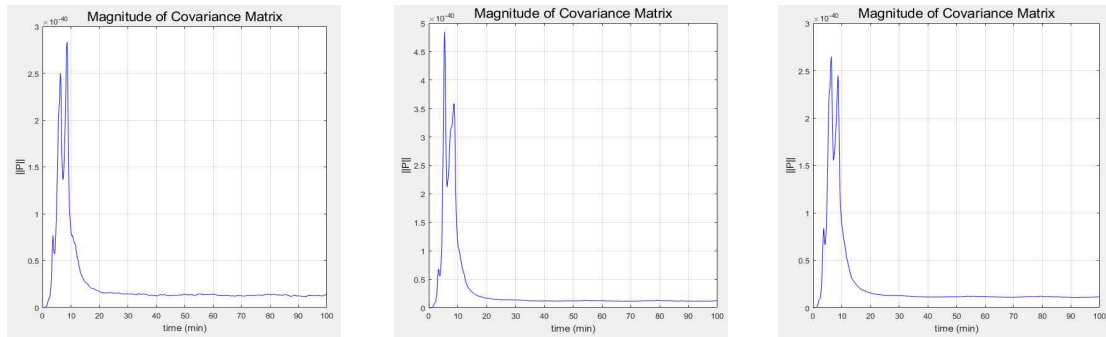
그 다음으로는 EKF에 의한 자세 error 변화에 대해 살펴보았다. 실행 후 10분까지는 초기에 설정한 error 180°에서 안정을 찾는 단계로 생각할 수 있으며 20분 이후에는 20°를 넘지 않는 수치를 유지하며 점차 0에 가깝게 줄어드는 모습을 볼 수 있다. 하지만 solar cell noise가 $\sigma_{solar} = 15^\circ$ 정도로 크게 설정된 값이기 때문에 정확도를 크게 신뢰할 수 없는 것을 유념해야할 것으로 보인다.

따라서 이번에도 angular velocity에서와 마찬가지로 solar cell noise를 $\sigma_{solar} = 10^\circ$ 와 5° 로 감소시키며 변화를 살펴보았다. 각도가 감소함에 따라 20분 이후에 발생하는 자세 error의 수치 및 oscillation의 폭이 0에 가깝게 감소하였고 5° 로 설정하였을 때는 눈에 띄게 0에 수렴하고 있음을 확인할 수 있다. 따라서 σ_{solar} 의 변화에 따른 자세 error가 Angular velocity에 비해 영향을 많이 받는 것으로 볼 수 있다.





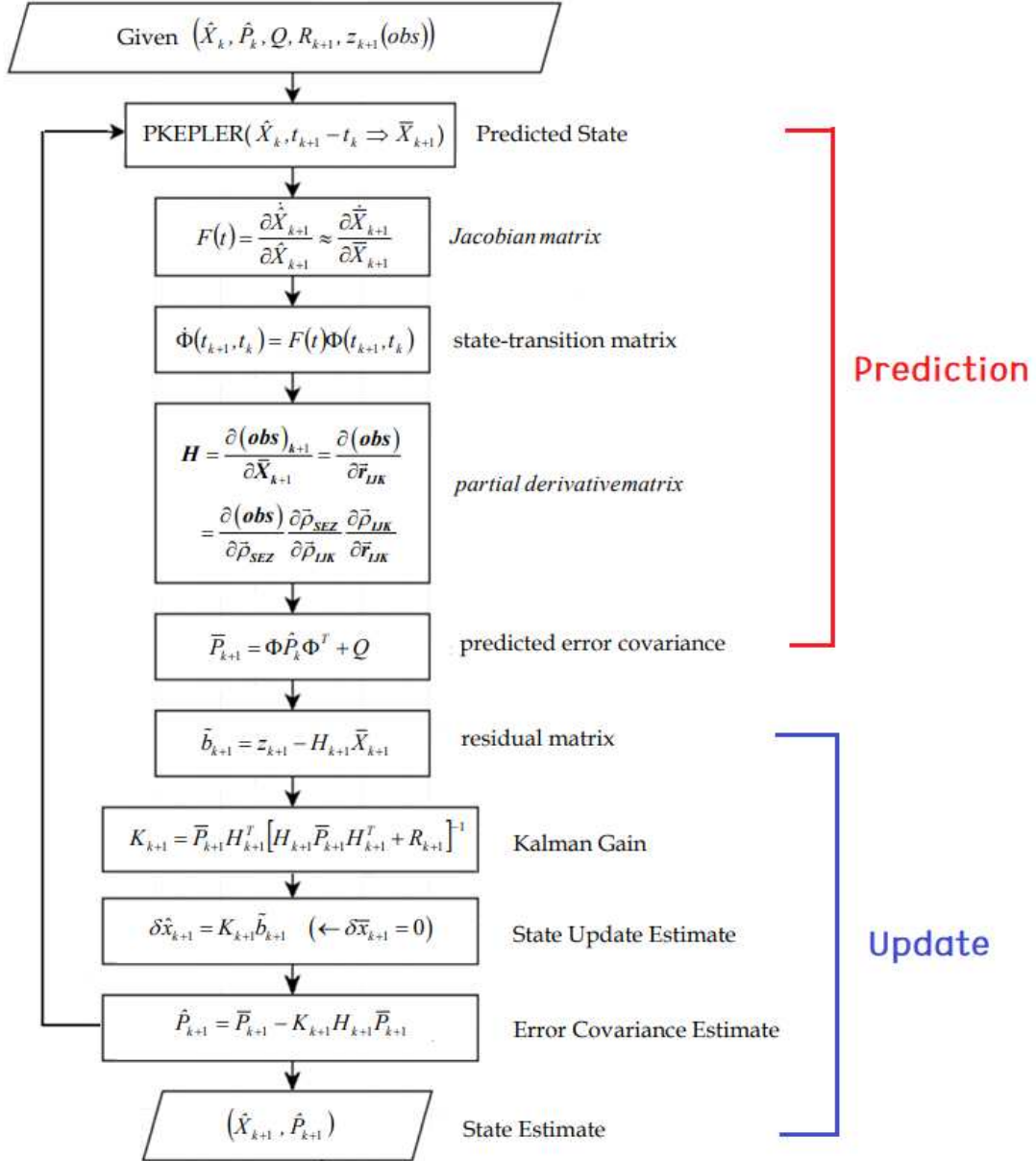
다음으로 Euler Angle의 변화를 살펴보았다. 위와 마찬가지로 σ_{solar} 값을 감소시키며 진행하였으며 세 경우에서 모두 simulation을 실행한 지 약 15분 후에 실제 값과 추정 값이 대체적으로 일치하는 모습을 보였다. 하지만 각도가 감소할수록 안정성을 찾는 자세 error와는 다르게 15°에서 5°로 감소함에 따라 실제 값과 추정 값에 차이가 발생하였다.



마지막으로 추정 값에 대한 정확도를 나타내는 Covariance Matrix를 살펴보았다. 자세 error의 그래프 양상과 마찬가지로 15~20분 구간을 기준으로 불안정과 안정한 영역으로 나뉘었고 해당 시각의 이전 구간은 시도할 때 마다 큰 차이를 보여주며 σ_{solar} 에 따라 뚜렷한 상관관계를 찾기 어려웠다. 하지만 세 경우에서 모두 그 이후 구간에는 다른 parameter의 결과들과 마찬가지로 0에 가까워지다 일정하게 유지되는 모습을 볼 수 있었다. 이는 자세 결정의 불확실성이 적다는 것을 의미하므로 성공적인 filtering이 이루어졌다고 볼 수 있다.

위의 모든 결과에서 동일하게 약 20분정도의 충분히 시간이 흐른 후에야 실제 값과 추정된 값이 비슷한 양상을 유지하는 것을 알 수 있었다. 이 시간은 초기 자세 error의 추정 값 180°에서 안정화까지 소요되는 시간을 의미한다. 이러한 오차를 보이는 이유는 Euler angle이 -180°에서 180°로 한정되어 있기 때문에 각도가 360°씩 증가 또는 감소하는 것처럼 보이기 때문이다. 이런 현상을 무시한다면 20분 이후로는 자세 결정이 안정화되는 것을 볼 수 있다. 따라서 관측 데이터가 부족하고 관측 기간이 길다면 Kalman filter를 이용하기에는 적합하지 않는 것으로 보인다.

2) Make a flow chart of the EKF for attitude determination.



3) Explain the codes used in detail.

이번 과제를 Simulation 하는데 이용한 모든 코드 중, 중요도가 높다고 생각되는 10개의 코드를 선정해보았으며 이에 대한 설명을 간략하게 기술하였다. (알파벳 순서대로 나열)

- ① BDipole.m : 기울어진 쌍극자 모델(tilted dipole model)을 사용하여 자기장을 계산하는 코드로 계산한 자기장 벡터는 지구 중심 좌표계를 기준으로 표현되어 있다.
- ② CosD.m : 코사인에 대한 각도를 radian에서 degree로 변환하는 함수이다.
- ③ constant : main.m 코드에 사용되는 각종 상수에 대한 변수를 선언하는 함수이다.
- ④ EarthRot.m : 지구 중심 좌표계에서 지구 고정 좌표계로 변환하는 회전 행렬을 계산하는 함수이다.
- ⑤ func_F.m : quaternion & angular momentum을 linearized dynamic model로 반환하는 함수이다.
- ⑥ GMSTime.m : Julian date에서 GMST로 변환하는 함수이다.
- ⑦ main.m : 최종 X, P를 추정하는 함수이다. [자세한 설명은 문제 1) 참고]
- ⑧ SinD.m : 사인에 대한 각도를 radian에서 degree로 변환하는 함수이다.
- ⑨ SunV1.m : 지구 중심 관성계에서 표현한 인공위성에 대한 태양의 위치 벡터를 생성하는 코드로 위성과 태양까지의 거리와 시선 벡터를 출력한다.
- ⑩ Unit.m : 단위를 변환하는 함수이다.

4) Derive and explain all the equations used.

Equation 설명을 위해 수업시간에 공부한 필기본을 첨부하였다. 각각에 대해 도출하는 순서를 숫자로 표현하였고 이에 대한 설명은 사진 아래 입력하였다.

Extended Kalman Filter (EKF)

Given: $\hat{X}_k, \hat{P}_k, Q, R_{k+1}, Z_{k+1}(\text{obs}) \rightarrow \hat{X}_{k+1}, \hat{P}_{k+1}$

$\downarrow$ LKF에서는 0
EKF에서는 현재 시각에서의

$H = \frac{\partial g(x)}{\partial x} \bigg|_{\bar{x}_{k+1}}$ ④

Prediction ①

propa
 $P_{k+1|k}(\hat{X}_k, (t_{k+1} - t_k) \Rightarrow \bar{x}_{k+1})$ ② new trajectory

$F(t) = \frac{\partial \dot{x}_{k+1}}{\partial x_{k+1}} \approx \frac{\partial \dot{x}_{k+1}}{\partial \bar{x}_{k+1}} = \frac{\partial f(x)}{\partial x} \bigg|_{\bar{x}_{k+1}}$

$\Phi(t_{k+1}, t_k) = F(t) \Phi(t_{k+1}, t_k)$ ③

$\downarrow \Phi(t_{k+1}, t_k) \quad \delta \bar{x}_{k+1} = 0$
predicted state update

update ⑥

$\tilde{b}_{k+1} = Z_{k+1} - H_{k+1} \bar{x}_{k+1}$ ⑦

$K_{k+1} = \bar{P}_{k+1} H_{k+1}^T [H_{k+1} \bar{P}_{k+1} H_{k+1}^T + R_{k+1}]^{-1}$ ⑤ kalman gain

$\delta \hat{x}_{k+1} = K_{k+1} \tilde{b}_{k+1}$ ⑧ state update estimate

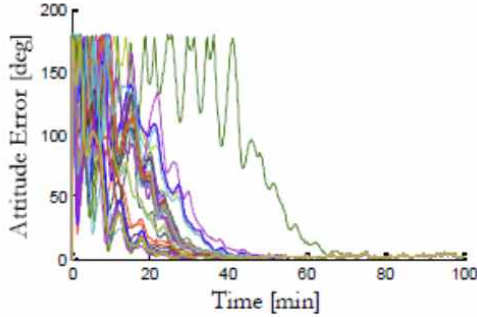
$\hat{P}_{k+1} = \bar{P}_{k+1} - K_{k+1} H_{k+1} \bar{P}_{k+1}$: Error Covariance Estimate ⑨

$\hat{X}_{k+1} = \bar{x}_{k+1} + \delta \hat{x}_{k+1}$: state Estimate ... 반복 ...

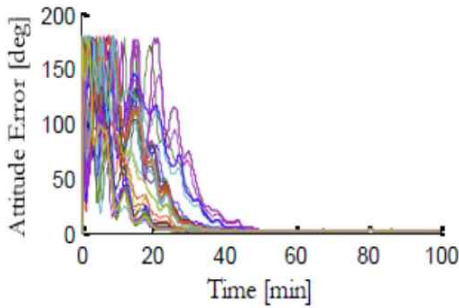
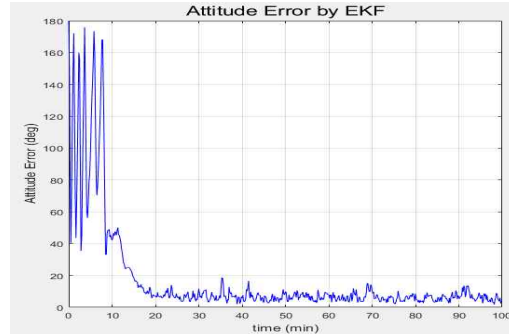
$\bar{P}_{k+1} = \Phi \hat{P}_k \Phi^T + Q$
predicted error covariance

- ① $\hat{X}_k$ 를 이용해 $\bar{x}_{k+1}$ 를 추정한다.)
- ② $\bar{x}_{k+1}$ 를 이용하여 Jacobian matrix (F)를 계산한다.
- ③ Jacobian 과 수치적분을 이용해 State-transition matrix (Φ)를 계산한다.
- ④ $\bar{x}_{k+1}$ 과 관측 모델을 이용하여 Partial derivative matrix (H)를 계산한다.
- ⑤ 위에서 계산한 state-transition matrix 와 이전의 covariance matrix 를 이용하여 현재의 covariance matrix ($\bar{P}_{k+1}$)를 추정한다.
- ⑥ 실제 관측, 계산된 관측 값의 차이를 통해 residual ($\tilde{b}_{k+1}$) 계산한다.
- ⑦ 추정한 covariance matrix 와 partial derivative matrix 를 통해 Kalman gain(K_{k+1})을 계산한다.
- ⑧ Karman gain 과 residual 을 이용하여 state vector 의 변화량($\delta \hat{x}_{k+1}$)을 계산한다.
- ⑨ $\hat{X}_{k+1}, \hat{P}_{k+1}$ 를 추정하고 Iteration 이 관측 데이터를 초과하게 되면 최종적인 covariance matrix 와 state vector 를 반복하여 계산한다.

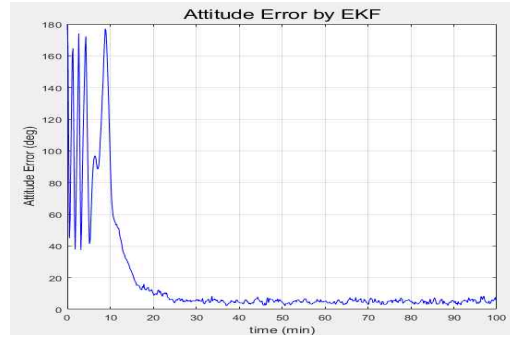
5) Compare your results and those in the paper, and explain your results.



($\sigma_{solar}=10^\circ$)



($\sigma_{solar}=5^\circ$)



위와 같이 논문의 attitude error 결과와 직접 simulation을 통해 얻은 결과를 비교해보았으며 Solar cell noise는 논문과 같이 순서대로 10° , 5° 로 설정하였다. 비교적 단순한 그래프가 출력된 결과에 비해 논문의 결과에서는 48개의 attitude error가 복잡하게 나타났다. 이에 따라, 적어도 20분이 지난 시점에는 자세가 안정화 되는 simulation 결과에 비해 논문의 결과에서는 이를 초과한 시점에서도 noise를 보이는 attitude error를 다수 확인할 수 있었다. 또한, 논문에서는 Converged pointing의 값이 각각 4.5° , 2.5° 로 계산되었지만 직접 수행해 본 코드에서는 각각 4.4722° 와 4.1527° 로 계산되었다. Solar cell noise의 값이 감소됨에 따라 converged pointing 값 또한 감소하는 것은 두 결과에서 모두 동일했으나, $\sigma_{solar}=10^\circ$ 일 때는 pointing의 값이 유사한 것에 비해 $\sigma_{solar}=5^\circ$ 일 때는 오차가 다소 발생하였다.

오차의 요인으로는 변수의 설정, 자기장 및 IGRF 등에 의한 기타 noise, 선형화에 의한 요인 등 다양하게 작용한다고 볼 수 있었지만 시간 간격에 의한 영향이 가장 크다고 생각해 보았다. 과제에서 사용한 시간 간격은 전체 궤도주기 5800초에 대해 6000초로 설정하였고 10초 간격으로 600개의 구간으로 나누어 X와 P를 추정하였다. 따라서 시간 간격이 매우 크기 때문에 이를 이용해 선형화하면 더 큰 오차가 발생할 것이라고 판단했다.

6) Improve the results by 최성문 in 2019

Table 2. Worst Case Disturbance Torques	
Disturbance Torque	Magnitude [N · m]
Aerodynamic	$6.97 \text{ e} - 9$
Gravity Gradient	$1.75 \text{ e} - 9$
Solar Pressure	$1.64 \text{ e} - 9$

논문에서 제시한 worst case에 대한 disturbance(table 2)를 반영하기 위해 Aerodynamic, Gravity Gradient, Solar Pressure 등에 대한 Torques를 입력함으로서 더 정밀한 자세결정이 될 수 있을 것이다. X, t, r_true 등의 dynamic에 대한 함수가 포함된 코드(dynamics.m)에 이를 추가할 수 있다. 또한, 위성 진행방향의 반대쪽에 대해 대기에 의한 영향 그리고, Suv-vector 방향에 대해 태양풍에 의한 영향을 고려하면 오차를 줄이고 더욱 정밀한 attitude로 결과 값을 향상시킬 수 있다.